

Практическая работа №2

Задание:

1. Функции высшего порядка + ООП

```
2. import 'dart:math';
3.
4. class MathProcessor {
5.   final List<double> numbers;
6.
7.   // 1. Конструктор принимает список чисел List<double>
8.   MathProcessor(this.numbers);
9.
10.  // 2. Метод process принимает функцию-обработчик и возвращает новый
    список
11.  List<double> process(double Function(double) processor) {
12.    List<double> result = [];
13.    for (double num in numbers) {
14.      result.add(processor(num)); // Применяем переданную функцию к числу
15.    }
16.    return result;
17.  }
18.
19.  // 3. Статический метод генерирует список случайных чисел дробного типа
20.  static List<double> getRandomNumbers(int count, {double max = 100.0}) {
21.    final random = Random();
22.    return List.generate(count, (_) => random.nextDouble() * max);
23.  }
24.}
25.
26.void main() {
27.  print('--- Генерация случайных чисел ---');
28.  // Генерируем 5 случайных чисел
29.  List<double> randomList = MathProcessor.getRandomNumbers(5, max: 10.0);
30.
31.  // Создаем объект обработчика
32.  MathProcessor processor = MathProcessor(randomList);
33.
34.  print('Исходный список:');
35.  print(processor.numbers.map((n) => n.toStringAsFixed(2)).toList());
36.
37.  print('\n--- Тест 1: Возведение в квадрат (анонимная функция) ---');
38.  List<double> squared = processor.process((num) => num * num);
39.  print(squared.map((n) => n.toStringAsFixed(2)).toList());
40.
41.  print('\n--- Тест 2: Умножение на 10 (стрелочная функция) ---');
42.  List<double> multiplied = processor.process((num) => num * 10);
43.  print(multiplied.map((n) => n.toStringAsFixed(2)).toList());
44.}
45.
```

task2.1.dart X

 task2.1.dart > ...

```

1 import 'dart:math';
2
3 class MathProcessor {
4     final List<double> numbers;
5
6     // 1. Конструктор принимает список чисел List<double>
7     MathProcessor(this.numbers);
8
9     // 2. Метод process принимает функцию-обработчик и возвращает новый список
10    List<double> process(double Function(double) processor) {
11        List<double> result = [];
12        for (double num in numbers) {
13            result.add(processor(num)); // Применяем переданную функцию к числу
14        }
15        return result;
16    }
17
18    // 3. Статический метод генерирует список случайных чисел дробного типа
19    static List<double> getRandomNumbers(int count, {double max = 100.0}) {
20        final random = Random();
21        return List.generate(count, (_) => random.nextDouble() * max);
22    }
23 }
24

```

Run | Debug

```
25 void main() {
26     print('--- Генерация случайных чисел ---');
27     // Генерируем 5 случайных чисел
28     List<double> randomList = MathProcessor.getRandomNumbers(5, max: 10.0);
29
30     // Создаем объект обработчика
31     MathProcessor processor = MathProcessor(randomList);
32
33     print('Исходный список:');
```

+ v  powershell

```
PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task2.1.dart
--- Генерация случайных чисел ---
Исходный список:
[1.99, 4.12, 0.82, 1.75, 8.58]

--- Тест 1: Возведение в квадрат (анонимная функция) ---
[3.97, 16.96, 0.67, 3.05, 73.65]

--- Тест 2: Умножение на 10 (стрелочная функция) ---
[19.92, 41.18, 8.18, 17.47, 85.82]
PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> |
```

2. Абстрактные классы + Наследование

```
import 'dart:math';

// 1. Абстрактный класс Shape
abstract class Shape {
  // Абстрактный метод для вычисления площади
  double area();

  // 3. Фабричный конструктор для создания Circle или Rectangle на основе JSON
  factory Shape.fromJson(Map<String, dynamic> json) {
    final type = json['type'];

    if (type == 'circle') {
      // Приведение к double на случай, если в JSON придет int (например, 5
      // вместо 5.0)
      return Circle((json['radius'] as num).toDouble());
    } else if (type == 'rectangle') {
      return Rectangle(
        (json['width'] as num).toDouble(),
        (json['height'] as num).toDouble(),
      );
    } else {
      throw ArgumentError('Неизвестный тип фигуры: $type');
    }
  }
}

// 2. Класс Circle, наследующий Shape
class Circle implements Shape {
  final double radius;

  Circle(this.radius);

  @override
  double area() {
    return pi * radius * radius;
  }
}

// 2. Класс Rectangle, наследующий Shape
class Rectangle implements Shape {
  final double width;
  final double height;

  Rectangle(this.width, this.height);

  @override
  double area() {
    return width * height;
  }
}
```


3. Статические методы + Константы

```
class PasswordValidator {
    // 1. Константные требования к паролю
    static const int minLength = 8;

    // Регулярное выражение для проверки наличия хотя бы одной цифры
    static final RegExp _digitRegExp = RegExp(r'\d');

    // 2. Статический метод для проверки пароля
    static bool validate(String password) {
        // Проверка длины пароля
        if (password.length < minLength) {
            return false;
        }

        // Проверка наличия хотя бы одной цифры
        if (!_digitRegExp.hasMatch(password)) {
            return false;
        }

        return true; // Пароль прошёл все проверки
    }
}

void main() {
    print('--- Тестирование PasswordValidator ---');

    List<String> testPasswords = [
        'Aezakmi',      // Меньше 8 символов, нет цифры (false)
        'Qwertuiop',    // Больше 8 символов, но нет цифры (false)
        '1234567',      // Меньше 8 символов, есть цифры (false)
        'Password123',  // Больше 8 символов, есть цифры (true)
        'TestPass321',  // Больше 8 символов, есть цифры (true)
    ];

    for (var pass in testPasswords) {
        bool isValid = PasswordValidator.validate(pass);
        print('Пароль: "$pass" -> ${isValid ? "Корректный ✓" : "Некорректный ✗"}');
    }
}
```

```
task2.1.dart task2.2.dart task2.3.dart X
task2.3.dart > main
1 class PasswordValidator {
2     // 1. Константные требования к паролю
3     static const int minLength = 8;
4
5     // Регулярное выражение для проверки наличия хотя бы одной цифры
6     static final RegExp _digitRegExp = RegExp(r'\d');
7
8     // 2. Статический метод для проверки пароля
9     static bool validate(String password) {
10         // Проверка длины пароля
11         if (password.length < minLength) {
12             return false;
13         }
14
15         // Проверка наличия хотя бы одной цифры
16         if (!_digitRegExp.hasMatch(password)) {
17             return false;
18         }
19
20         return true; // Пароль прошёл все проверки
21     }
22 }
23
24 Run | Debug
25 void main() {
26     print('--- Тестирование PasswordValidator ---');
27
28     List<String> testPasswords = [
29         'Aezakmi', // Меньше 8 символов, нет цифры (false)
30         'Qwertyuiop', // Больше 8 символов, но нет цифры (false)
31         '1234567', // Меньше 8 символов, есть цифры (false)
32         'Password123', // Больше 8 символов, есть цифры (true)
33         'TestPass321', // Больше 8 символов, есть цифры (true)
34     ];
35 }
36
37 ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ POSTMAN CONSOLE + v powershell
PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task2.3.dart
--- Тестирование PasswordValidator ---
Пароль: "Aezakmi" -> Некорректный X
Пароль: "Qwertyuiop" -> Некорректный X
Пароль: "1234567" -> Некорректный X
Пароль: "Password123" -> Корректный ✓
Пароль: "TestPass321" -> Корректный ✓
PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> 
```

4. Банковская система

```
import 'dart:math';

// Класс Банковский счет
class BankAccount {
  final String accountNumber;
  double balance;
  String ownerName;

  // Конструктор: balance по умолчанию = 0, accountNumber генерируется случайно
  BankAccount(this.ownerName)
    : balance = 0.0,
      accountNumber = _generateAccountNumber();

  // Приватный метод для генерации случайного 10-значного номера счета
  static String _generateAccountNumber() {
    final random = Random();
    String number = '';
    for (int i = 0; i < 10; i++) {
      number += random.nextInt(10).toString();
    }
    return number;
  }

  // Пополнение счета
  void deposit(double amount) {
    if (amount <= 0) {
      print('Ошибка: сумма пополнения должна быть больше нуля.');
```

```
      return;
    }
    balance += amount;
    print('Счет $accountNumber пополнен на $amount руб. Текущий баланс: $balance
руб.');
```

```
  }

  // Снятие денег
  bool withdraw(double amount) {
    if (amount <= 0) {
      print('Ошибка: сумма снятия должна быть больше нуля.');
```

```
      return false;
    }
    if (amount > balance) {
      print('Ошибка: недостаточно средств на счете $accountNumber (доступно:
$balance руб.).'
```

```
    );
    return false;
  }
  balance -= amount;
  print('Со счета $accountNumber снято $amount руб. Текущий баланс: $balance
руб.');
```

```
  return true;
}
```

```

    }

    // Вывод текущего баланса
    void displayBalance() {
        print('Владелец: $ownerName | Счет: $accountNumber | Баланс: $balance руб.');
```

```
    }
}
```

```
// Класс Банк
```

```
class Bank {
```

```
    List<BankAccount> accounts = [];
```

```
    // Создает новый счет и добавляет его в список
```

```
    void addAccount(String ownerName) {
```

```
        final newAccount = BankAccount(ownerName);
```

```
        accounts.add(newAccount);
```

```
        print('Создан новый счет для пользователя $ownerName (№ $newAccount.accountNumber).');
```

```
    }
```

```
    // Поиск счета по номеру (возвращает счет или null)
```

```
    BankAccount? findAccount(String accountNumber) {
```

```
        for (var account in accounts) {
```

```
            if (account.accountNumber == accountNumber) {
```

```
                return account;
```

```
            }
```

```
        }
```

```
        return null;
```

```
    }
```

```
    // Перевод денег между счетами
```

```
    void transfer(String fromAccountNumber, String toAccountNumber, double amount) {
```

```
        if (amount <= 0) {
```

```
            print('Ошибка перевода: сумма должна быть больше нуля.');
```

```
            return;
```

```
        }
```

```
        final fromAccount = findAccount(fromAccountNumber);
```

```
        final toAccount = findAccount(toAccountNumber);
```

```
        if (fromAccount == null) {
```

```
            print('Ошибка перевода: счет отправителя $fromAccountNumber не найден.');
```

```
            return;
```

```
        }
```

```
        if (toAccount == null) {
```

```
            print('Ошибка перевода: счет получателя $toAccountNumber не найден.');
```

```
            return;
```

```
        }
```

```

        // Если получилось успешно снять деньги со счета отправителя, то зачисляем
        получателю
        if (fromAccount.withdraw(amount)) {
            toAccount.balance += amount; // Зачисляем напрямую без вывода лишних
            принтов deposit
            print('Успешный перевод: $amount руб. со счета $fromAccountNumber на счет
            $toAccountNumber.');
```

task2.1.darttask2.2.darttask2.3.darttask2.4.dart X

task2.4.dart > BankAccount > _generateAccountNumber

```
1 import 'dart:math';
2
3 // Класс Банковский счет
4 class BankAccount {
5     final String accountNumber;
6     double balance;
7     String ownerName;
8
9     // Конструктор: balance по умолчанию = 0, accountNumber генерируется случайно
10    BankAccount(this.ownerName)
11        : balance = 0.0,
12          accountNumber = _generateAccountNumber();
13
14    // Приватный метод для генерации случайного 10-значного номера счета
15    static String _generateAccountNumber() {
16        final random = Random();
17        String number = '';
18        for (int i = 0; i < 10; i++) {
19            number += random.nextInt(10).toString();
20        }
21        return number;
22    }
23
24    // Пополнение счета
25    void deposit(double amount) {
26        if (amount <= 0) {
27            print('Ошибка: сумма пополнения должна быть больше нуля.');
```

ПРОБЛЕМЫВЫХОДНЫЕ ДАННЫЕКОНСОЛЬ ОТЛАДКИТЕРМИНАЛПОРТЫPOSTMAN CONSOLE+ powershell

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task2.4.dart

--- Тестирование банковской системы ---

Создан новый счет для пользователя Алексей (№ 0476782834).

Создан новый счет для пользователя Мария (№ 5326887908).

Проверка баланса изначально ---

Владелец: Алексей | Счет: 0476782834 | Баланс: 0.0 руб.

Владелец: Мария | Счет: 5326887908 | Баланс: 0.0 руб.

Операции со счетом Алексея ---

Счет 0476782834 пополнен на 5000.0 руб. Текущий баланс: 5000.0 руб.

Со счета 0476782834 снято 1500.0 руб. Текущий баланс: 3500.0 руб.

Ошибка: недостаточно средств на счете 0476782834 (доступно: 3500.0 руб.).

Операция перевода Марии ---

Со счета 0476782834 снято 2000.0 руб. Текущий баланс: 1500.0 руб.

Успешный перевод: 2000.0 руб. со счета 0476782834 на счет 5326887908.

Итоговое состояние счетов ---

Владелец: Алексей | Счет: 0476782834 | Баланс: 1500.0 руб.

Владелец: Мария | Счет: 5326887908 | Баланс: 2000.0 руб.

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2>

5. Игра «РПГ-персонажи»

```
import 'dart:math';

// 1. Абстрактный класс Character
abstract class Character {
  String name;
  int health = 100;
  int level = 1;

  Character(this.name, {this.health = 100, this.level = 1});

  // Абстрактные методы
  int attack();
  void defend(int damage);

  // Общий метод
  bool isAlive() {
    return health > 0;
  }
}

// 2.а Класс Warrior (Воин)
class Warrior extends Character {
  Warrior(String name, {int level = 1}) : super(name, level: level);

  @override
  int attack() {
    return 15 + level * 2;
  }

  @override
  void defend(int damage) {
    // Защита: уменьшает урон на 20%
    int actualDamage = (damage * 0.8).round();
    health -= actualDamage;
    if (health < 0) health = 0;
    print('$name (Воин) заблокировал 20% урона. Получено урона: $actualDamage. Осталось HP: $health');
  }
}

// 2.б Класс Mage (Маг)
class Mage extends Character {
  Mage(String name, {int level = 1}) : super(name, level: level);

  @override
  int attack() {
    return 10 + level * 3;
  }
}
```

```

@Override
void defend(int damage) {
    // Защита: получает полный урон, но вылечивается на 5 HP
    health -= damage;
    if (health > 0) {
        health += 5;
        if (health > 100) health = 100; // Здоровье не может быть выше начального
        print('$name (Маг) получил полный урон $damage, но исцелился на 5 HP.
Осталось HP: $health');
    } else {
        health = 0;
        print('$name (Маг) получил урон $damage и погиб.');
```

```

    }
}

// 2.в Класс Rogue (Разбойник)
class Rogue extends Character {
    final Random _random = Random();

    Rogue(String name, {int level = 1}) : super(name, level: level);

    @Override
    int attack() {
        int baseDamage = 12 + level * 2; // Базовый урон для баланса игры
        // 20% шанс на критический удар (x3)
        if (_random.nextInt(100) < 20) {
            print(' КРИТИЧЕСКИЙ УДАР от $name!');
            return baseDamage * 3;
        }
        return baseDamage;
    }

    @Override
    void defend(int damage) {
        // Защита: уворот (50% шанс избежать урона)
        if (_random.nextBool()) {
            print(' $name (Разбойник) уклонился от атаки! Урон не получен. Осталось HP:
$health');
        } else {
            health -= damage;
            if (health < 0) health = 0;
            print('$name (Разбойник) не успел уклониться. Получено урона: $damage.
Осталось HP: $health');
        }
    }
}

// Дополнительно: Симуляция боя между двумя персонажами
void startBattle(Character p1, Character p2) {
    print('\n!!! БИТВА НАЧИНАЕТСЯ: ${p1.name} VS ${p2.name} !!!\n');
```

```

int round = 1;

while (p1.isAlive() && p2.isAlive()) {
    print('--- Раунд $round ---');

    // Ход первого персонажа
    int damageP1 = p1.attack();
    print('${p1.name} атакует на $damageP1 урона. ');
    p2.defend(damageP1);

    if (!p2.isAlive()) {
        print('\n ${p1.name} побеждает в бою!');
        break;
    }

    print(''); // Разделитель ходов

    // Ход второго персонажа
    int damageP2 = p2.attack();
    print('${p2.name} атакует на $damageP2 урона. ');
    p1.defend(damageP2);

    if (!p1.isAlive()) {
        print('\n ${p2.name} побеждает в бою!');
        break;
    }

    round++;
    print('=' * 40);
}

}

void main() {
    // Создаем персонажей 3-го уровня
    Warrior boris = Warrior('Борис', level: 3);
    Mage sorceress = Mage('Волшебница', level: 3);
    Rogue vitaliy = Rogue('Виталий', level: 3);

    // передаем в бой реально созданных персонажей (например, Бориса и Виталия)
    startBattle(boris, vitaliy);
}

```

task2.1.dart task2.2.dart task2.3.dart task2.4.dart task2.5.dart 1 X

task2.5.dart > main

127 }
128
Run | Debug
129 void main() {
130 // Создаем персонажей 3-го уровня
131 Warrior boris = Warrior('Борис', level: 3);
132 Mage sorceress = Mage('Волшебница', level: 3);
133 Rogue vitality = Rogue('Виталий', level: 3);
134
135 // передаем в бой реально созданных персонажей (например, Бориса и Виталия)
136 startBattle(boris, vitality);
137 }
138
139

ПРОБЛЕМЫ 1 ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ POSTMAN CONSOLE + v powershell [] [] [] []

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task2.5.dart

!!! БИТВА НАЧИНАЕТСЯ: Борис VS Виталий !!!

--- Раунд 1 ---
Борис атакует на 21 урона.
Виталий (Разбойник) уклонился от атаки! Урон не получен. Осталось HP: 100

КРИТИЧЕСКИЙ УДАР от Виталий!
Виталий атакует на 54 урона.
Борис (Воин) заблокировал 20% урона. Получено урона: 43. Осталось HP: 57
=====

--- Раунд 2 ---
Борис атакует на 21 урона.
Виталий (Разбойник) уклонился от атаки! Урон не получен. Осталось HP: 100

Виталий атакует на 18 урона.
Борис (Воин) заблокировал 20% урона. Получено урона: 14. Осталось HP: 43
=====

--- Раунд 3 ---
Борис атакует на 21 урона.
Виталий (Разбойник) уклонился от атаки! Урон не получен. Осталось HP: 100

КРИТИЧЕСКИЙ УДАР от Виталий!
Виталий атакует на 54 урона.
Борис (Воин) заблокировал 20% урона. Получено урона: 43. Осталось HP: 0

Виталий побеждает в бою!

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> []